

DOM (Document Object Model) ek tree structure hai jo HTML document ko represent karta hai. JavaScript se DOM elements select karne ke liye **querySelector** (first match) aur **querySelectorAll** (all matches, NodeList) use karte hain — CSS selectors ki tarah syntax. DOM traversal se hum elements ke relationships (parent, child, sibling) navigate kar sakte hain.

```
// querySelector – first match
const title = document.querySelector('h1');
const firstBtn = document.querySelector('.btn'); // first .btn class
const userCard = document.querySelector('#user-123'); // by id
const input = document.querySelector('input[type="email"]'); // attribute selector
const nested = document.querySelector('.card .title'); // descendant selector
// querySelectorAll – all matches (NodeList, NOT Array!)
const allBtns = document.querySelectorAll('.btn');
const allItems = document.querySelectorAll('li.item');
// NodeList has forEach, but NOT map/filter/reduce directly
allBtns.forEach(btn => btn.style.color = 'blue');
// Convert NodeList to Array for map/filter/reduce
const btnArray = [...allBtns]; // spread operator
const btnArray2 = Array.from(allBtns); // Array.from
const texts = [...allBtns].map(b => b.textContent);
```

```
// DOM TRAVERSAL
const card = document.querySelector('.card');

// PARENT
card.parentElement; // direct parent element
card.closest('.container'); // nearest ancestor matching selector (or itself)

// CHILDREN
card.children; // HTMLCollection of element children (no text nodes)
card.childNodes; // NodeList – includes text nodes, comments too
```

```
card.firstChild; // first child element
card.lastElementChild; // last child element

// SIBLINGS
card.nextElementSibling; // next sibling element
card.previousElementSibling; // previous sibling element

// Real example: highlight current nav item's siblings
const navItems = document.querySelectorAll('.nav-item');
navItems.forEach(item => {
  item.addEventListener('click', () => {
    // Remove 'active' from all siblings
    item.parentElement.querySelectorAll('.nav-item')
      .forEach(sib => sib.classList.remove('active'));
    item.classList.add('active');
  });
});

// closest() – find nearest ancestor (very useful for event delegation)
document.addEventListener('click', (e) => {
  const card = e.target.closest('.card'); // works even if clicked on child
  if (card) console.log('Clicked card:', card.dataset.id);
});
```

■ **Critical: children vs childNodes vs HTMLCollection vs NodeList**

children = HTMLCollection (elements only). childNodes = NodeList (text+comments+elements too).

querySelectorAll returns STATIC NodeList — DOM badlega to NodeList update nahi hoga.

getElementsByClassName returns LIVE HTMLCollection — DOM change se automatically update hota hai.

querySelector(All) modern aur preferred — but performance-critical loops mein getElementById faster hai.

children = HTMLCollection (elements only). childNodes = NodeList (text+comments+elements too).

querySelectorAll returns STATIC NodeList — DOM badlega to NodeList update nahi hoga.

getElementsByClassName returns LIVE HTMLCollection — DOM change se automatically update hota hai.

querySelector(All) modern aur preferred — but performance-critical loops mein getElementById faster hai.

Topic 2 — createElement, appendChild, innerHTML vs textContent

DOM mein naye elements **create** karne aur **insert** karne ke multiple tarike hain. **innerHTML** HTML string parse karta hai (XSS risk!), **textContent** plain text set karta hai (safe). Modern methods: **append**, **prepend**, **before**, **after** — more flexible than appendChild.

[illegible]

```
// ■■■ CREATE & APPEND ■■■  
  
// Method 1: createElement + set properties + appendChild  
const li = document.createElement('li');  
li.textContent = 'New Task';  
li.className = 'task-item';  
li.dataset.id = '101'; // data-id="101" attribute  
document.querySelector('ul').appendChild(li);  
  
// Method 2: Building complex structure  
function createTaskCard(task) {  
  const card = document.createElement('div');  
  card.className = 'task-card';  
  card.dataset.id = task.id;
```

```

const title = document.createElement('h3');
title.textContent = task.title;

const btn = document.createElement('button');
btn.textContent = 'Delete';
btn.className = 'delete-btn';

card.append(title, btn); // append multiple nodes at once!
return card;
}

document.querySelector('.task-list').append(createTaskCard({id:1, title:'Buy milk'}));

// Modern insertion methods (better than appendChild)
parent.append(child1, child2, 'text node too!'); // end, multiple
parent.prepend(child); // beginning
element.before(newEl); // insert before element
element.after(newEl); // insert after element
element.replaceWith(newEl); // replace element entirely
element.remove(); // remove element directly (no parent.removeChild needed!)

```

innerHTML — Parses HTML (risky)

```

// innerHTML – parses as HTML
const div = document.querySelector('#out');

div.innerHTML = '<b>Bold</b> text';
// Renders: BOLD text (bold!)

// ■ XSS RISK with user input!
const userInput = '<img src=x
onerror=alert(1)>';
div.innerHTML = userInput; // ■ DANGEROUS!
// Executes malicious script!

// innerHTML re-parses ENTIRE subtree
// – loses event listeners, slower
div.innerHTML += '<li>New item</li>';
// ■ Re-renders ALL children, not just new!

```

textContent — Plain Text (safe)

```

// textContent – plain text, SAFE
const div = document.querySelector('#out');

div.textContent = '<b>Bold</b> text';
// Renders literal text: <b>Bold</b> text
// (not bold – shows the tags as text)

// ■ SAFE with user input
const userInput = '<img src=x
onerror=alert(1)>';
div.textContent = userInput; // ■ SAFE!
// Displays as text, doesn't execute

// For HTML structure – use createElement
// or sanitize with DOMPurify library
// before using innerHTML with user data

```

■ Critical: innerHTML XSS Vulnerability

User-generated content (comments, search, profile bio) ko NEVER directly innerHTML mein daalo.

Attacker <script> ya onerror= attributes inject kar sakta hai — cookies steal, redirect, etc.

Safe alternatives: textContent (plain text), createElement (structured), DOMPurify.sanitize() (if HTML needed).

Rule: agar data user se aaya hai → textContent. Agar aap khud HTML likh rahe ho → innerHTML OK.

Topic 3 — Event Listeners, Bubbling & Capturing

addEventListener se element pe event handler attach karte hain. Jab event fire hota hai, JS teen phases follow karta hai: **Capturing** (root → target), **Target** (element pe), **Bubbling** (target → root). By default listeners bubbling phase mein chalte hain.

addEventListener Basics


```
});  
inner.addEventListener('click', () => console.log('never runs'));
```

■ Real World: Modal Close on Outside Click

Bubbling use karke modal ke bahar click pe close karo:

```
document.addEventListener('click', e => { if(!modal.contains(e.target)) closeModal(); })
```

`e.stopPropagation()` modal ke andar click pe lagao — taaki ye outer listener trigger na ho.

Topic 4 — Event Delegation Pattern

Event Delegation ek pattern hai jisme hum parent element pe EK listener lagate hain — child elements ke individual listeners ki jagah. Bubbling ka use karke pata karte hain ki kaunsa child clicked hua (`e.target`).
Faayde: performance (1 listener vs 1000), aur **dynamically added elements** automatically kaam karte hain — naya listener add karne ki zaroorat nahi.

■ Without Delegation (broken for dynamic items)

```
// ■ WITHOUT Delegation  
// Add listener to EACH item  
document.querySelectorAll('.todo-item')  
  .forEach(item => {  
    item.querySelector('.delete')  
      .addEventListener('click', () => {  
        item.remove();  
      });  
  });  
  
// PROBLEM: new items added later  
// (e.g. via addTodo()) WON'T have  
// the delete listener!  
function addTodo(text) {  
  const li = createTodoEl(text);  
  list.appendChild(li);  
  // ■ forgot to add listener!  
}
```

■ With Delegation (scales perfectly)

```
// ■ WITH Delegation  
// ONE listener on parent – works  
// for ALL items, even future ones!  
const list =  
  document.querySelector('.todo-list');  
  
list.addEventListener('click', (e) => {  
  const deleteBtn = e.target.closest('.delete');  
  if (deleteBtn) {  
    deleteBtn.closest('.todo-item').remove();  
    return;  
  }  
  
  const checkbox = e.target.closest('.toggle');  
  if (checkbox) {  
    checkbox.closest('.todo-item')  
      .classList.toggle('done');  
  }  
});  
  
// New items automatically work!  
function addTodo(text) {  
  list.appendChild(createTodoEl(text));  
  // ■ No extra listener needed!  
}
```

Event Delegation — Table Actions

```
// Full event delegation example – Table row actions  
const table = document.querySelector('#data-table');  
table.addEventListener('click', (e) => {  
  const row = e.target.closest('tr');  
  if (!row) return;
```

```
const id = row.dataset.id;

if (e.target.matches('.edit-btn')) {
  console.log('Edit row', id);
} else if (e.target.matches('.delete-btn')) {
  if (confirm('Delete this row?')) row.remove();
} else if (e.target.matches('.view-btn')) {
  console.log('View details for', id);
}

});

// matches() – check if element matches a selector
// closest() – find nearest ancestor (or self) matching selector
// Both essential for event delegation!
```

■ Critical: When NOT to Use Delegation

mouseenter/mouseleave events DON'T bubble — delegation nahi chalega (use mouseover/mouseout instead, with care).

Focus/blur events — bubble nahi karte by default (use focusin/focusout for delegation).

Bahut deep nested structures — `e.target.closest()` thoda slow ho sakta hai (negligible for most apps).

Topic 5 — LocalStorage & SessionStorage

Web Storage API se browser mein key-value data store kar sakte hain — **localStorage** permanent hai (tab close hone ke baad bhi rehta hai), **sessionStorage** sirf tab/session ke liye hai. Dono sirf STRINGS store karte hain — objects/arrays ke liye JSON.stringify/parse use karna padta hai.

localStorage Persistent — browser band karne ke baad bhi data rehta hai. ~5-10MB limit.	sessionStorage Tab/session-only — tab close hone par clear ho jaata hai. Same API.	JSON methods Storage strings only store karta hai — objects ko stringify/parse karna padta hai.
---	--	---

LocalStorage API & JSON Pattern

[illegible]

■ Critical: LocalStorage Limitations & Security

Sync API — large data se main thread block ho sakta hai. IndexedDB use karo bade datasets ke liye.

Storage limit ~5-10MB per domain — browser dependent.

NEVER store sensitive data (passwords, tokens with long expiry) — XSS se accessible hai.

JSON.parse() try/catch mein wrap karo — corrupted/manually-edited data se crash ho sakta hai.

Day 5 Learning Outcomes

querySelector/querySelectorAll aur DOM traversal confidently use kar paoge.

Elements create, insert, aur safely update (textContent vs innerHTML) kar paoge.

Event bubbling/capturing samajh kar predict kar paoge ki kaunsa listener pehle chalega.

Event delegation pattern se dynamic lists efficiently handle kar paoge.

LocalStorage se data persist karke To-Do app jaisi apps bana paoge.